

МИНОБРНАУКИ РОССИИ
МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(МАИ)

ОТЧЁТ
О ЛАБОРАТОРНОЙ РАБОТЕ
по теме:
АЛГОРИТМЫ КВАДРАТИЧНОЙ СОРТИРОВКИ И АНАЛИЗ
ЭФФЕКТИВНОСТИ

СПИСОК ИСПОЛНИТЕЛЕЙ

Преподаватель

подпись, дата

Павлов О.В.

Студент группы

МЗО-113БВ-25

подпись, дата

Садунян Р.К.

1 Цель работы

Цель работы — реализовать и сравнить различные варианты квадратичных сортировок, научиться измерять временные и операционные затраты, а также анализировать зависимость эффективности алгоритмов от размера массива и структуры исходных данных.

2 Описание алгоритмов

В работе реализованы три алгоритма сортировки по возрастанию:

1. пузырьковая сортировка с флагом;
2. шейкерная сортировка;
3. сортировка чёт-нечет.

Для каждого алгоритма измеряются количество сравнений элементов, количество обменов элементов и время работы в микросекундах. Перед каждым измерением исходный массив копируется в рабочий массив с помощью `memcpy`, поэтому один запуск алгоритма не влияет на последующие.

Размеры массивов фиксированы:

$$n \in \{10, 100, 500, 1000, 2000, 5000, 10000\}. \quad (1)$$

Значения элементов являются целыми числами из диапазона $[-32000; 32000]$. Типы исходных данных хранятся в `enum class DataType` и принимают значения:

- `random` — случайные числа;
- `nearly_sorted` — отсортированный массив, после чего выполняется 10% случайных обменов;
- `reversed` — массив в обратном порядке.

2.1 Пузырьковая сортировка с флагом

Пузырьковая сортировка с флагом выполняет последовательные проходы по массиву и сравнивает соседние элементы. Если левый элемент больше правого, элементы меняются местами. После каждого прохода максимальный элемент оставшейся неотсортированной части смещается вправо.

В отличие от классического варианта, алгоритм хранит флаг `swapped`. Если за проход не было ни одного обмена, массив уже отсортирован и дальнейшие проходы не нужны.

Сложность:

- лучший случай: $O(n)$;
- средний случай: $O(n^2)$;
- худший случай: $O(n^2)$.

2.2 Шейкерная сортировка

Шейкерная сортировка является двунаправленным вариантом пузырьковой сортировки. Сначала выполняется проход слева направо, который переносит максимальный элемент текущего диапазона в конец. Затем выполняется проход справа налево, который переносит минимальный элемент текущего диапазона в начало.

После каждой пары проходов левая и правая границы рабочей области сужаются. Такой подход эффективнее обычного пузырька на данных, где мелкие элементы находятся ближе к концу массива.

Сложность:

- лучший случай: $O(n)$;
- средний случай: $O(n^2)$;
- худший случай: $O(n^2)$.

2.3 Сортировка чёт-нечет

Сортировка чёт-нечет выполняет чередующиеся фазы сравнения соседних пар. На нечётной фазе проверяются пары с индексами (1, 2), (3, 4) и так далее. На чётной фазе проверяются пары (0, 1), (2, 3) и так далее.

Фазы повторяются до тех пор, пока за полный цикл не произойдёт ни одного обмена. Алгоритм близок к пузырьковой сортировке по идее локального перемещения элементов, но организует сравнения по фазам.

Сложность:

- лучший случай: $O(n)$;
- средний случай: $O(n^2)$;
- худший случай: $O(n^2)$.

3 Блок-схемы

В этом разделе приведены подробные блок-схемы реализованных сортировок и укрупнённая блок-схема основной программы.

4 Пузырьковая сортировка с флагом

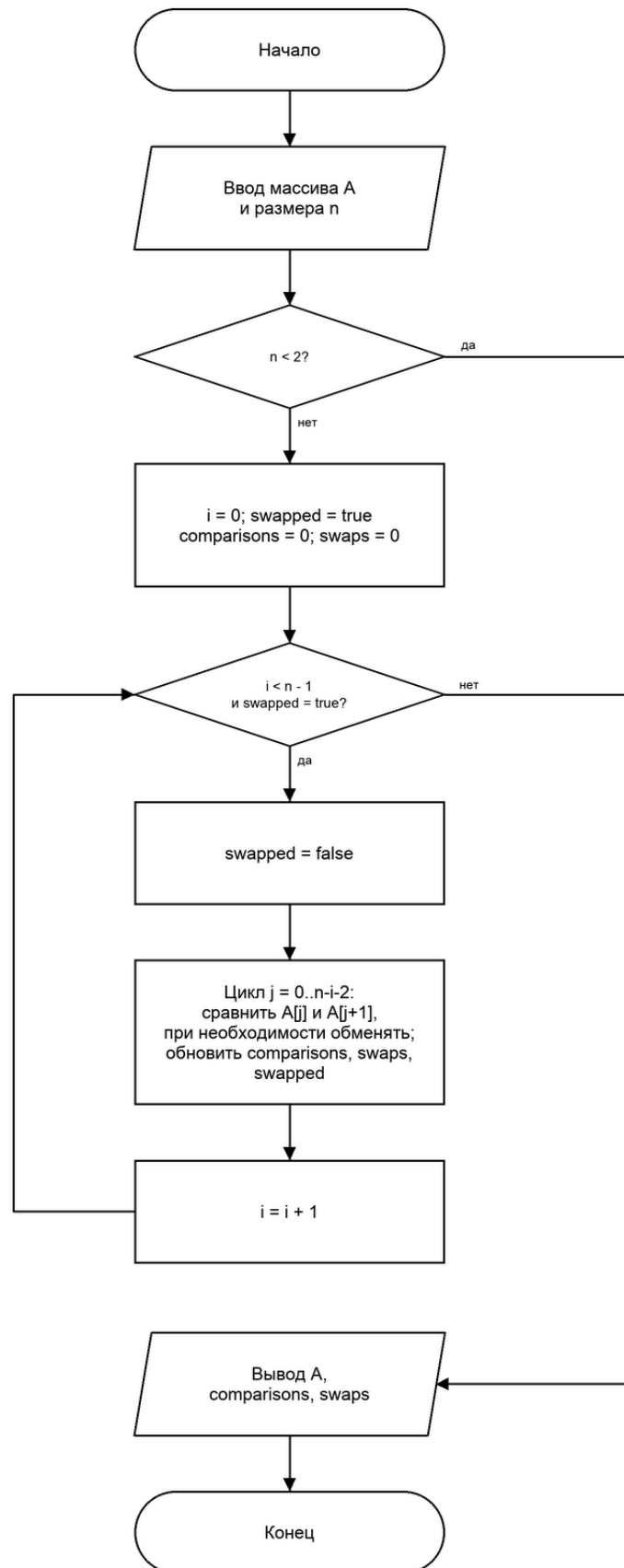


Рисунок 1 — Блок-схема пузырьковой сортировки с флагом

5 Шейкерная сортировка

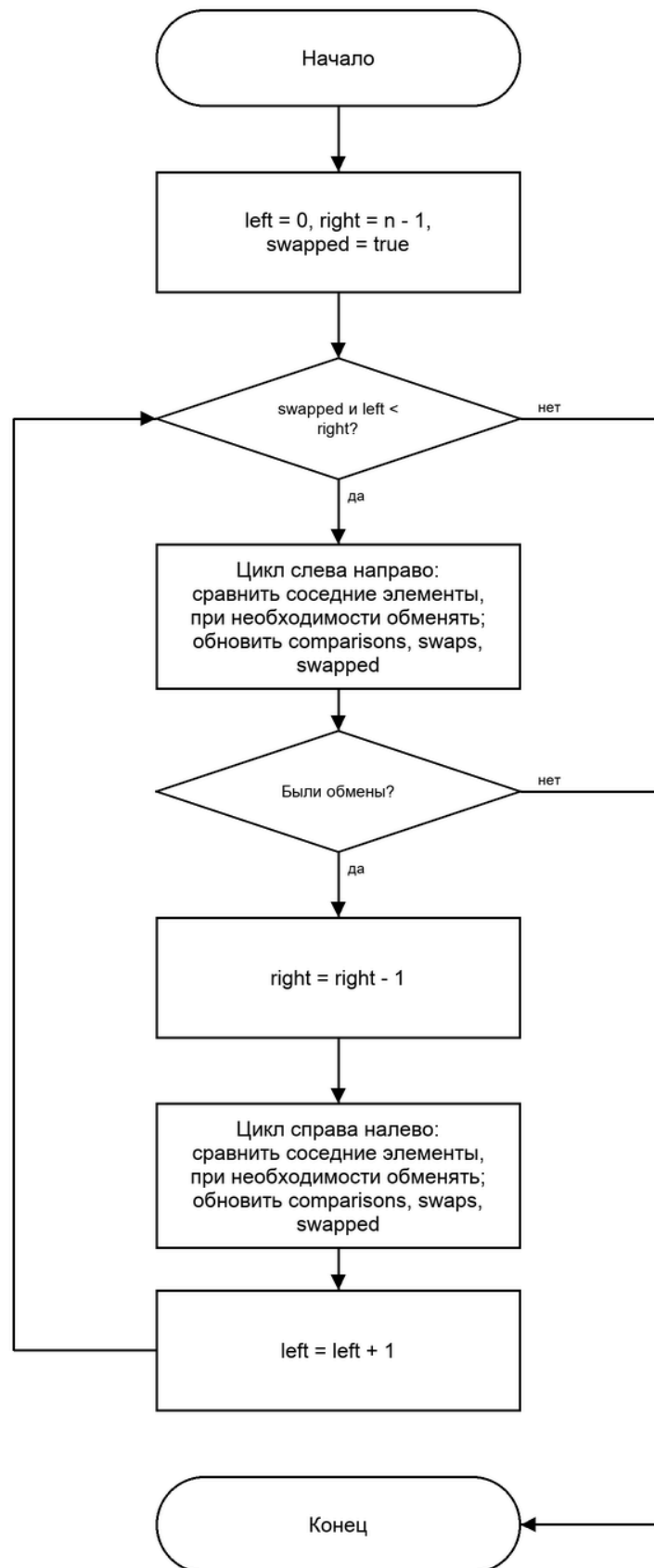


Рисунок 2 — Блок-схема шейкерной сортировки

5.1 Сортировка чёт-нечет

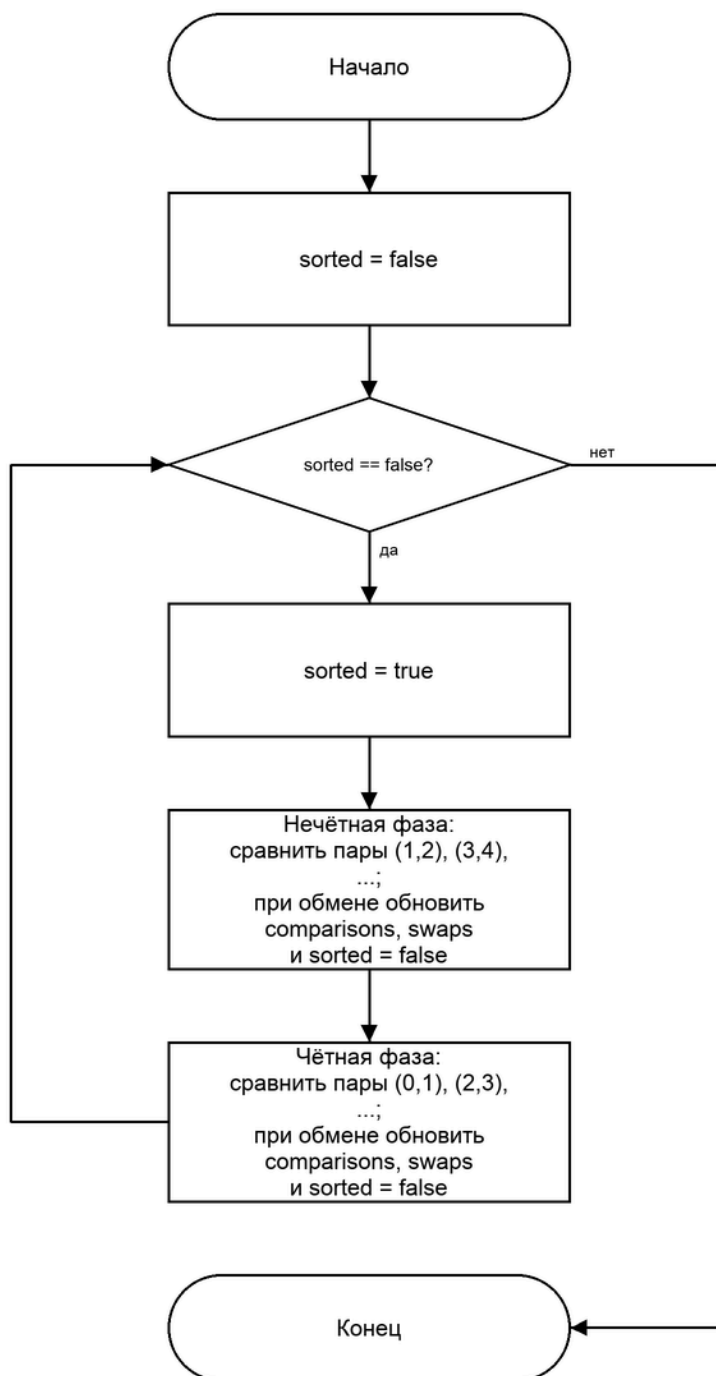


Рисунок 3 — Блок-схема сортировки чёт-нечет

5.2 Основная программа

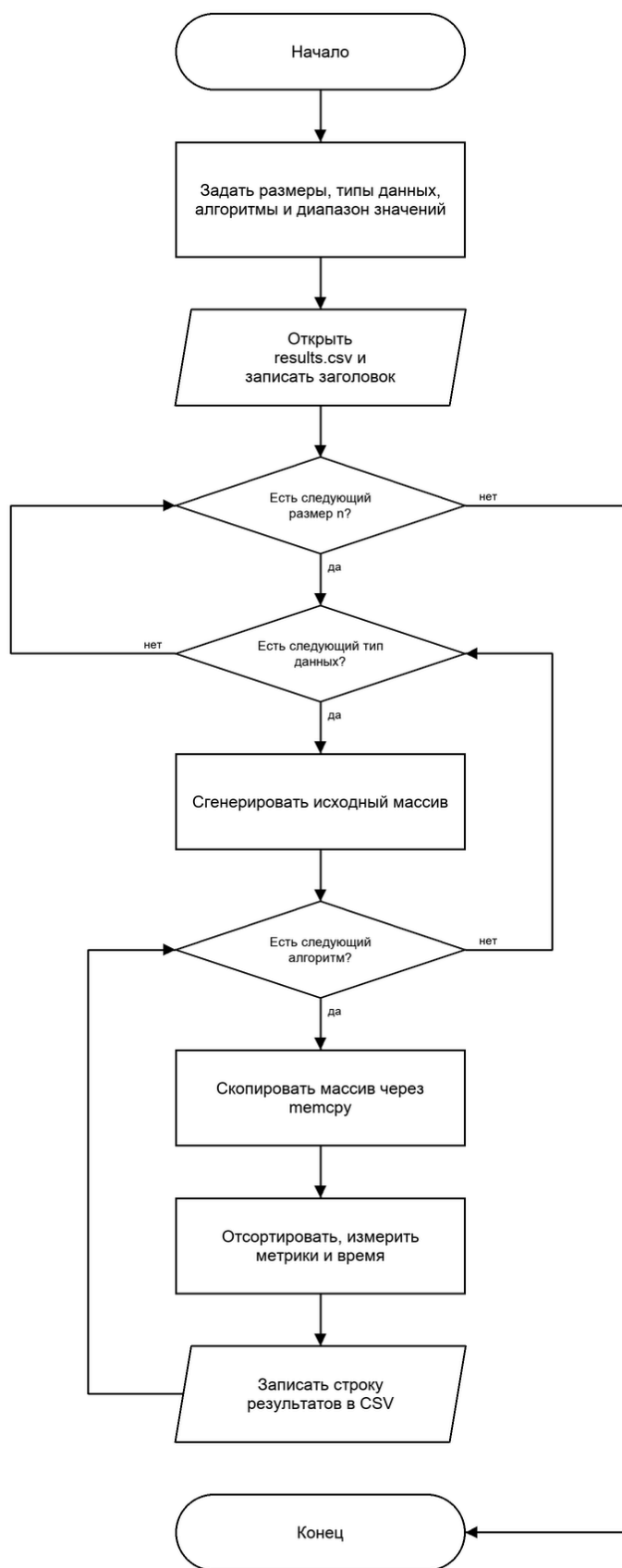


Рисунок 4 — Укрупнённая блок-схема основной программы

6 Результаты экспериментов

После запуска программы создаётся файл `results.csv` в кодировке UTF-8. Каждая строка соответствует одному сочетанию размера массива, типа исходных данных и алгоритма.

6.1 Таблица результатов

Случайные данные

Размер	Алгоритм	Сравнения	Обмены	Время, мкс
10	Пузырьковая с флагом	45	25	0
10	Шейкерная	39	25	0
10	Чёт-нечёт	54	25	0
100	Пузырьковая с флагом	4740	2402	7
100	Шейкерная	3960	2402	7
100	Чёт-нечёт	4653	2402	7
500	Пузырьковая с флагом	124540	64923	122
500	Шейкерная	99099	64923	103
500	Чёт-нечёт	119760	64923	106
1000	Пузырьковая с флагом	498797	252115	422
1000	Шейкерная	377235	252115	451
1000	Чёт-нечёт	482517	252115	384
2000	Пузырьковая с флагом	1997230	1009797	2940
2000	Шейкерная	1487434	1009797	1492
2000	Чёт-нечёт	1973013	1009797	1482
5000	Пузырьковая с флагом	12492940	6215706	12406
5000	Шейкерная	9416097	6215706	9163
5000	Чёт-нечёт	12262547	6215706	7907
10000	Пузырьковая с флагом	49992654	25003962	34874
10000	Шейкерная	37647035	25003962	33566
10000	Чёт-нечёт	49665033	25003962	26479

Почти отсортированные данные

Размер	Алгоритм	Сравнения	Обмены	Время, мкс
10	Пузырьковая с флагом	17	1	0
10	Шейкерная	17	1	0
10	Чёт-нечёт	18	1	0
100	Пузырьковая с флагом	4650	556	3
100	Шейкерная	855	556	1

Размер	Алгоритм	Сравнения	Обмены	Время, мкс
100	Чёт-нечёт	4950	556	5
500	Пузырьковая с флагом	123475	14391	72
500	Шейкерная	22824	14391	26
500	Чёт-нечёт	112774	14391	243
1000	Пузырьковая с флагом	496725	59623	390
1000	Шейкерная	101222	59623	90
1000	Чёт-нечёт	463536	59623	448
2000	Пузырьковая с флагом	1977055	236408	1712
2000	Шейкерная	388885	236408	577
2000	Чёт-нечёт	1791104	236408	2275
5000	Пузырьковая с флагом	12482965	1486924	7029
5000	Шейкерная	2234785	1486924	2179
5000	Чёт-нечёт	12072585	1486924	11755
10000	Пузырьковая с флагом	49868244	5884412	25274
10000	Шейкерная	8821725	5884412	7754
10000	Чёт-нечёт	47485251	5884412	43811

Обратный порядок

Размер	Алгоритм	Сравнения	Обмены	Время, мкс
10	Пузырьковая с флагом	45	45	0
10	Шейкерная	45	45	0
10	Чёт-нечёт	54	45	0
100	Пузырьковая с флагом	4950	4950	3
100	Шейкерная	4950	4950	5
100	Чёт-нечёт	5049	4950	2
500	Пузырьковая с флагом	124750	124746	118
500	Шейкерная	124750	124746	134
500	Чёт-нечёт	125249	124746	69
1000	Пузырьковая с флагом	499500	499492	413
1000	Шейкерная	499500	499492	1026
1000	Чёт-нечёт	500499	499492	518
2000	Пузырьковая с флагом	1999000	1998973	1993
2000	Шейкерная	1999000	1998973	2954
2000	Чёт-нечёт	2000999	1998973	1544
5000	Пузырьковая с флагом	12497500	12497311	7482
5000	Шейкерная	12497500	12497311	12934
5000	Чёт-нечёт	12502499	12497311	6239
10000	Пузырьковая с флагом	49995000	49994223	30139
10000	Шейкерная	49995000	49994223	52337
10000	Чёт-нечёт	50004999	49994223	24616

6.2 Графики результатов экспериментов

6.2.1 Время работы

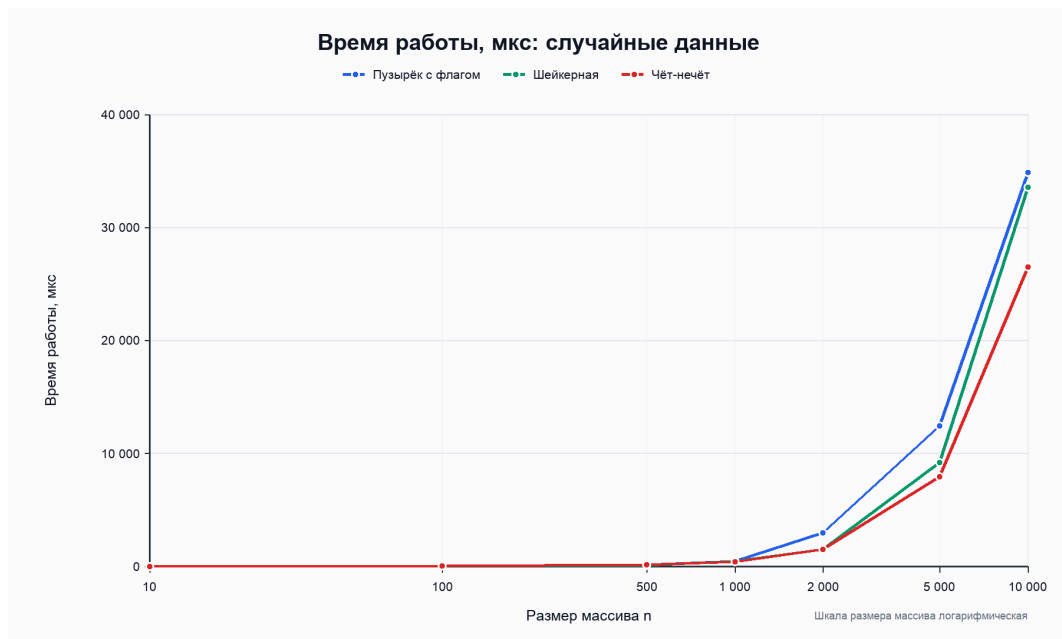


Рисунок 5 — Время работы алгоритмов на случайных данных



Рисунок 6 — Время работы алгоритмов на почти отсортированных данных

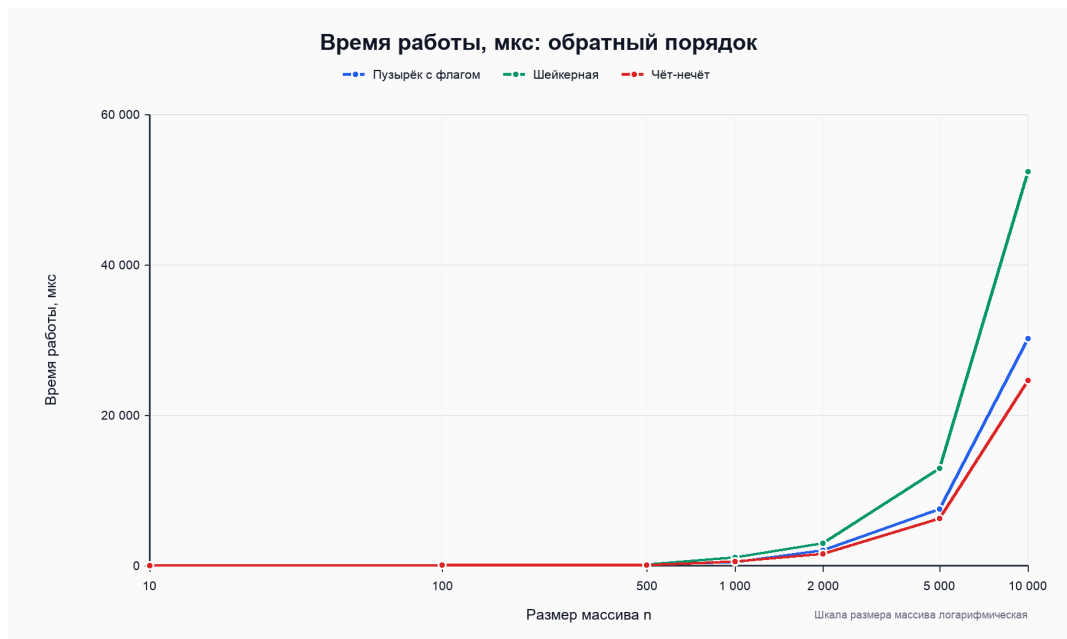


Рисунок 7 — Время работы алгоритмов на данных в обратном порядке

6.2.2 Количество сравнений элементов



Рисунок 8 — Количество сравнений на случайных данных

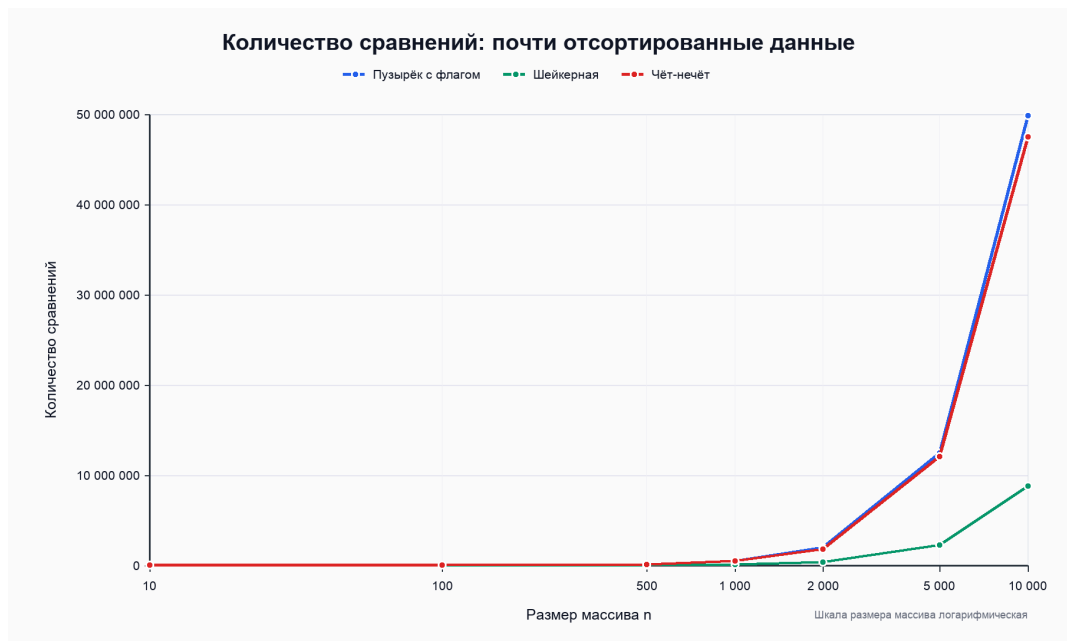


Рисунок 9 — Количество сравнений на почти отсортированных данных

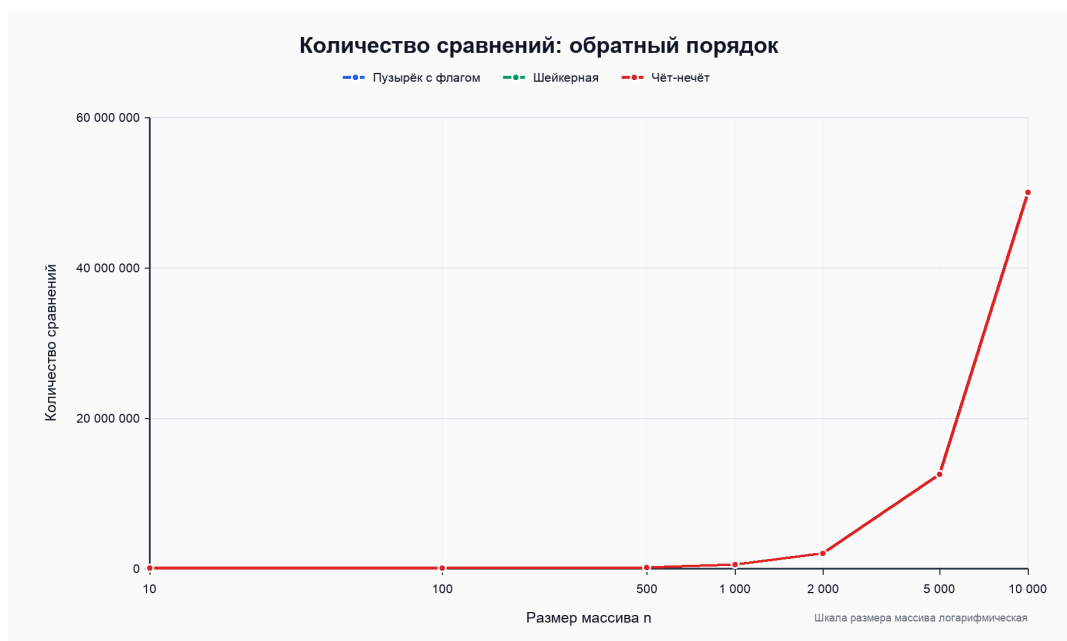


Рисунок 10 — Количество сравнений на данных в обратном порядке

6.2.3 Количество обменов элементов

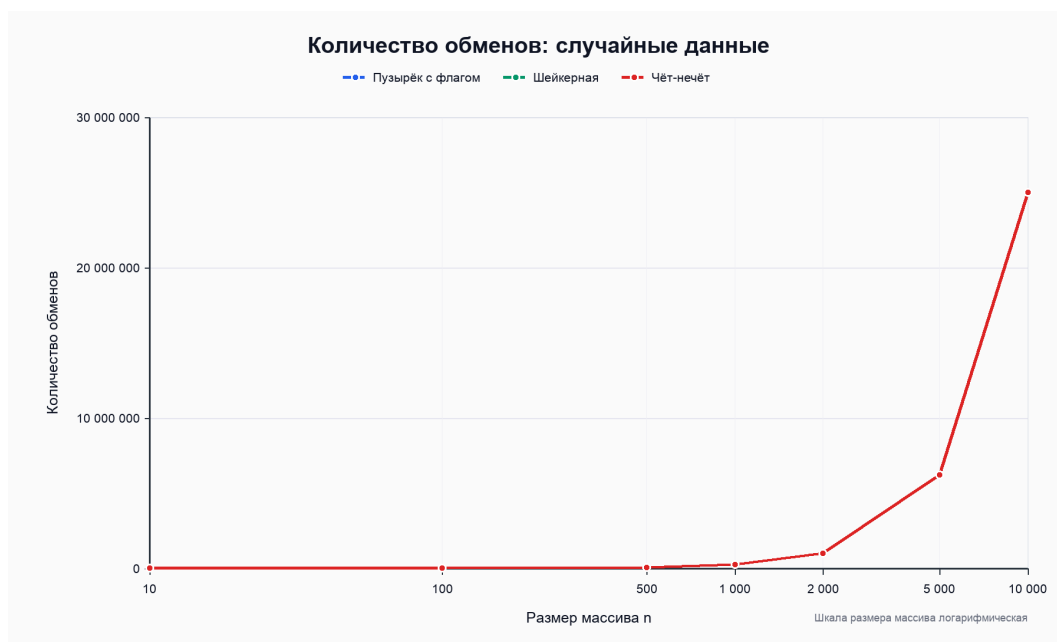


Рисунок 11 — Количество обменов на случайных данных

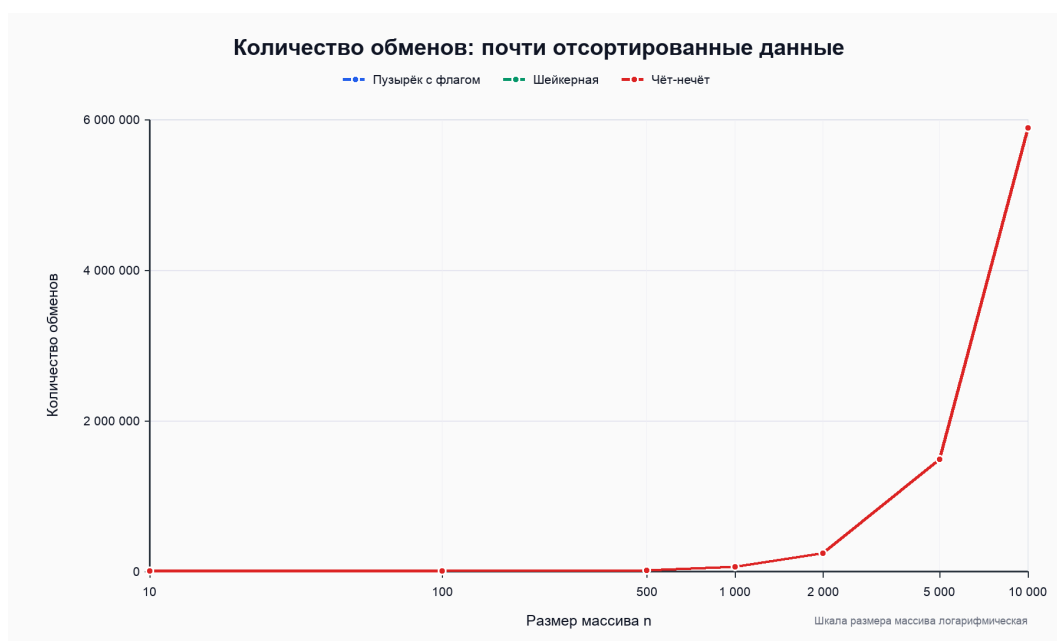


Рисунок 12 — Количество обменов на почти отсортированных данных



Рисунок 13 — Количество обменов на данных в обратном порядке

7 Анализ результатов

На случайных данных все три алгоритма демонстрируют квадратичный рост числа операций. При увеличении размера массива в несколько раз количество сравнений и обменов растёт значительно быстрее, чем сам размер массива. Это соответствует теоретической оценке $O(n^2)$ для среднего случая.

Пузырьковая сортировка с флагом хорошо реагирует на уже упорядоченную или почти упорядоченную структуру данных. Если на очередном проходе обменов нет, алгоритм завершает работу досрочно. Поэтому на почти отсортированных массивах число сравнений оказывается существенно меньше, чем в худшем случае, хотя случайные обмены всё равно могут заставить алгоритм выполнить несколько проходов.

Шейкерная сортировка также использует досрочное завершение и дополнительно движется в обе стороны. Это уменьшает рабочий диапазон с двух сторон и помогает быстрее возвращать малые элементы в начало массива. На случайных и почти отсортированных данных она часто выполняет меньше лишних проходов, чем однонаправленный пузырьк.

Сортировка чёт-нечёт выполняет попеременные фазы сравнения пар. На случайных и обратных данных она сохраняет квадратичный характер роста. Количество обменов близко к числу инверсий в массиве, а количество сравнений зависит от числа фаз, необходимых для полного упорядочивания.

На данных в обратном порядке все алгоритмы работают близко к худшему случаю. Такой массив содержит максимальное количество инверсий, поэтому требуется много обменов. Графики сравнений и обменов имеют форму, близкую к квадратичной зависимости.

8 Выводы по работе

В ходе лабораторной работы были реализованы три квадратичных алгоритма сортировки: пузырьковая сортировка с флагом, шейкерная сортировка и сортировка чёт-нечёт. Для каждого алгоритма были измерены количество сравнений, количество обменов и время выполнения.

Эксперимент подтвердил теоретическую оценку $O(n^2)$ для среднего и худшего случаев. Исходная упорядоченность данных заметно влияет на практическую эффективность: оптимизации с флагом и сужением границ позволяют быстрее завершать работу на почти отсортированных массивах. На обратном порядке преимущество оптимизаций уменьшается, потому что алгоритмам приходится исправлять максимальное количество инверсий.

9 Приложение. Исходный код программы

```
#include <algorithm>
#include <chrono>
#include <fstream>
#include <iostream>
#include <random>
#include <string>
#include <vector>

using namespace std;

struct Stats {
    long long comparisons = 0;
    long long swaps = 0;
    long long time_us = 0;
};

enum DataType {
    Random,
    NearlySorted,
    Reversed
};

enum Algorithm {
    BubbleFlagged,
    Shaker,
    OddEven
};

string toString(DataType type) {
    const string names[] = {"random", "nearly_sorted", "reversed"};
    return names[type];
}

string toString(Algorithm algorithm) {
    const string names[] = {"bubble_flagged", "shaker", "odd_even"};
    return names[algorithm];
}

vector<int> generateData(size_t n, DataType type, mt19937& rng) {
    uniform_int_distribution<int> valueDist(-32000, 32000);
```

```

vector<int> data(n);

for (size_t i = 0; i < n; ++i) {
    data[i] = valueDist(rng);
}

if (type == NearlySorted || type == Reversed) {
    sort(data.begin(), data.end());
}

if (type == NearlySorted && n > 1) {
    uniform_int_distribution<size_t> indexDist(0, n - 1);
    const size_t swapCount = max<size_t>(1, n / 10);

    for (size_t i = 0; i < swapCount; ++i) {
        const size_t left = indexDist(rng);
        const size_t right = indexDist(rng);
        swap(data[left], data[right]);
    }
} else if (type == Reversed) {
    reverse(data.begin(), data.end());
}

return data;
}

void bubbleFlagged(int* data, size_t n, Stats& stats) {
    bool swapped = true;

    for (size_t i = 0; i < n && swapped; ++i) {
        swapped = false;

        for (size_t j = 0; j + 1 < n - i; ++j) {
            ++stats.comparisons;
            if (data[j] > data[j + 1]) {
                swap(data[j], data[j + 1]);
                ++stats.swaps;
                swapped = true;
            }
        }
    }
}
}

```

```

void shakerSort(int* data, size_t n, Stats& stats) {
    if (n < 2) {
        return;
    }

    size_t left = 0;
    size_t right = n - 1;
    bool swapped = true;

    while (swapped && left < right) {
        swapped = false;

        for (size_t i = left; i < right; ++i) {
            ++stats.comparisons;
            if (data[i] > data[i + 1]) {
                swap(data[i], data[i + 1]);
                ++stats.swaps;
                swapped = true;
            }
        }

        if (!swapped) {
            break;
        }

        --right;
        swapped = false;

        for (size_t i = right; i > left; --i) {
            ++stats.comparisons;
            if (data[i - 1] > data[i]) {
                swap(data[i - 1], data[i]);
                ++stats.swaps;
                swapped = true;
            }
        }

        ++left;
    }
}

void oddEvenSort(int* data, size_t n, Stats& stats) {
    bool sorted = false;

```

```

while (!sorted) {
    sorted = true;

    for (size_t i = 1; i + 1 < n; i += 2) {
        ++stats.comparisons;
        if (data[i] > data[i + 1]) {
            swap(data[i], data[i + 1]);
            ++stats.swaps;
            sorted = false;
        }
    }

    for (size_t i = 0; i + 1 < n; i += 2) {
        ++stats.comparisons;
        if (data[i] > data[i + 1]) {
            swap(data[i], data[i + 1]);
            ++stats.swaps;
            sorted = false;
        }
    }
}

bool isSortedAscending(const vector<int>& data) {
    for (size_t i = 1; i < data.size(); ++i) {
        if (data[i - 1] > data[i]) {
            return false;
        }
    }
    return true;
}

Stats runSort(const vector<int>& source, Algorithm algorithm) {
    vector<int> working = source;

    Stats stats;
    const auto start = chrono::high_resolution_clock::now();

    switch (algorithm) {
        case BubbleFlagged:
            bubbleFlagged(working.data(), working.size(), stats);
            break;
    }
}

```

```

        case Shaker:
            shakerSort(working.data(), working.size(), stats);
            break;
        case OddEven:
            oddEvenSort(working.data(), working.size(), stats);
            break;
    }

    const auto finish = chrono::high_resolution_clock::now();
    stats.time_us =
        chrono::duration_cast<chrono::microseconds>(finish - start).count();

    if (!isSortedAscending(working)) {
        cerr << "Ошибка: массив не отсортирован алгоритмом "
             << toString(algorithm) << '\n';
    }

    return stats;
}

int main() {
    const vector<size_t> sizes = {10, 100, 500, 1000, 2000, 5000,
    10000};
    const vector<DataType> dataTypes = {Random, NearlySorted,
    Reversed};
    const vector<Algorithm> algorithms = {BubbleFlagged, Shaker,
    OddEven};

    mt19937 rng(42);
    ofstream output("results.csv");

    if (!output) {
        cerr << "Не удалось открыть файл results.csv\n";
        return 1;
    }

    output << "Size;DataType;Algorithm;Comparisons;Swaps;Time_us\n";

    for (size_t size : sizes) {
        for (DataType dataType : dataTypes) {
            const vector<int> source = generateData(size, dataType,
rng);

```

```

    for (Algorithm algorithm : algorithms) {
        const Stats stats = runSort(source, algorithm);

        output << size << ';'
                << toString(dataType) << ';'
                << toString(algorithm) << ';'
                << stats.comparisons << ';'
                << stats.swaps << ';'
                << stats.time_us << '\n';

        cout << "n=" << size
              << ", type=" << toString(dataType)
              << ", algorithm=" << toString(algorithm)
              << " done\n";
    }
}

cout << "Готово. Результаты записаны в results.csv\n";
return 0;
}

```

Листинг 1 — Исходный код программы для проведения экспериментов